Requirement Validation Document

DockWatch: Docker Container Health Monitoring System

Version 1.0 — May 2026 — Systems Engineering Team

Contents

1 Introduction

1.1 Purpose

This document validates the functional requirements defined in the Software Requirements Specification (SRS) for DockWatch v1.0. Each requirement is evaluated against standard validation criteria to identify gaps, ambiguities, or missing constraints before the development and testing phases begin.

1.2 Scope

This validation covers all six functional requirements (FR-01 through FR-06) from SRS Section 3.1. Non-functional requirements are referenced where they directly affect functional completeness.

1.3 Validation Criteria

Each requirement is assessed against the following four criteria:

- **Clear:** The requirement is unambiguous and has only one interpretation.
- **Testable:** The requirement can be verified through a defined test method.
- **Complete:** All necessary details (values, conditions, constraints) are present.
- **Consistent:** The requirement does not contradict other requirements or system constraints.

1.4 References

- DockWatch SRS v1.0, April 15, 2026
- IEEE Std 830-1998, *IEEE Recommended Practice for Software Requirements Specifications*
- ISO/IEC 25010:2011, *Systems and Software Quality Requirements and Evaluation*

2 Requirement Validation Summary

ID	Requirement (Summary)	Status	Issue
FR-01	Poll Docker Engine API for container PID, CPU, RAM	Valid	None
FR-02	Detect failures within 30 seconds	Valid	None
FR-03	Trigger recovery actions based on configurable rules	Partially Valid	Recovery trigger time not specified
FR-04	Store historical metrics; expose via REST API	Partially Valid	Data retention period not defined
FR-05	Centralized dashboard with real-time status and trends	Partially Valid	"Real-time" refresh interval not quantified
FR-06	YAML configuration and JSON telemetry export	Valid	Minor: YAML validation rules not specified

Table 1: Requirement Validation Summary

3 Detailed Requirement Analysis

3.1 FR-01 — Container Discovery and Metric Polling

Requirement: *"System shall continuously poll Docker Engine API to discover running containers and track PID, CPU, and RAM usage."*

Criterion	Result	Justification
Clear	Pass	Metrics to collect (PID, CPU, RAM) are explicitly named.
Testable	Pass	Verified by querying the API and confirming metric values are returned.
Complete	Pass	Polling target, data points, and scope are all defined. NFR-02 provides the performance bound (<5% CPU overhead).
Consistent	Pass	Consistent with NFR-02 (resource constraint) and NFR-05 (scalability to 50 containers).

Verdict: **Fully Valid** — No changes required.

3.2 FR-02 — Failure Detection

Requirement: *"System shall detect container failures, unresponsive states, or health check violations within 30 seconds."*

Criterion	Result	Justification
Clear	Pass	Three failure types are explicitly listed: crash, unresponsive state, health check violation.
Testable	Pass	The 30-second bound is measurable. Test: simulate a container crash and record detection timestamp.
Complete	Pass	Detection window (30s) is quantified. Aligns with the Key Performance Objective in SRS Section 1.1.
Consistent	Pass	Consistent with FR-03 (recovery) and UC-02 (failure use case).

Verdict: **Fully Valid** — No changes required.

3.3 FR-03 — Automated Recovery

Requirement: *"System shall automatically trigger predefined recovery actions (restart, isolate, or notify) based on configurable rules."*

Criterion	Result	Justification
Clear	Pass	Recovery actions (restart, isolate, notify) are explicitly enumerated.
Testable	Pass	Each action can be triggered in a test environment and verified.
Complete	Fail	**Gap:** The maximum time allowed between failure detection and recovery trigger is not specified. Without this, the requirement cannot be fully tested for timeliness. *Recommendation: Add a time constraint, e.g., "recovery action shall be triggered within 10 seconds of failure detection."*
Consistent	Pass	Consistent with FR-02 (detection) and UC-02 (failure scenario).

Verdict: **Partially Valid** — Missing recovery trigger time constraint.

3.4 FR-04 — Historical Metric Storage

Requirement: *"System shall store historical performance metrics and exposure data via REST API for capacity planning."*

Criterion	Result	Justification
Clear	Pass	Purpose (capacity planning) and access method (REST API) are stated.
Testable	Pass	Verified by querying the REST API for historical data and confirming correct JSON response.
Complete	Fail	**Gap:** No data retention period is defined. SRS Section 2.4 mentions "configurable TTL parameters" but no default or minimum value is given. *Recommendation: Specify a default retention period, e.g., "metrics shall be retained for a minimum of 30 days by default."*

Criterion	Result	Justification
Consistent	Pass	Consistent with UC-03 (QA engineer querying historical data) and NFR-07 (JSON payloads).

Verdict: **Partially Valid** — Data retention period must be defined.

3.5 FR-05 — Real-Time Dashboard

Requirement: *"System shall provide a centralized UI/UX dashboard displaying real-time status, alerts, and historical trends."*

Criterion	Result	Justification
Clear	Pass	Dashboard content (status, alerts, historical trends) is listed.
Testable	Fail	**Gap:** "Real-time" is not quantified. Without a defined refresh interval, there is no measurable pass/fail criterion. *Recommendation: "Dashboard shall refresh container status every 5 seconds via WebSocket."*
Complete	Fail	The communication mechanism is not specified in this requirement. WebSocket is mentioned in SRS Section 1.2 and NFR-07 but not explicitly linked here.
Consistent	Pass	Consistent with NFR-01 (API response time) and NFR-07 (WebSocket streaming).

Verdict: **Partially Valid** — "Real-time" must be quantified with a specific refresh interval.

3.6 FR-06 — Configuration and Telemetry Export

Requirement: *"System shall support configuration management via YAML files and export telemetry in JSON format."*

Criterion	Result	Justification
Clear	Pass	Both formats (YAML for config, JSON for telemetry) are explicitly stated.

Criterion	Result	Justification
Testable	Pass	Verified by loading a YAML config and confirming the system applies it; and by calling the telemetry endpoint and validating JSON output.
Complete	Pass	Formats are defined. Minor gap: YAML schema is not documented, but acceptable at SRS level.
Consistent	Pass	Consistent with NFR-07 (interoperability) and UC-01 (admin configures via YAML).

Verdict: **Mostly Valid** — No blocking issues. YAML schema documentation recommended for design phase.

4 Consolidated Findings and Recommendations

4.1 Issues Found

ID	Issue	Recommended Fix
FR-03	Recovery trigger time not specified	Add: "recovery action triggered within 10 seconds of detection"
FR-04	Data retention period undefined	Add: "default retention of 30 days; configurable via TTL"
FR-05	"Real-time" not quantified	Add: "dashboard updates every 5 seconds via WebSocket"

Table 8: Validation Issues and Recommendations

4.2 Overall Assessment

Out of six functional requirements:

- **3 requirements** (FR-01, FR-02, FR-06) are fully valid and require no changes.
- **3 requirements** (FR-03, FR-04, FR-05) are partially valid and require minor additions before the design phase is finalized.
- **0 requirements** are invalid or contradictory.

The SRS is of good quality overall. The identified gaps are minor and can be resolved by adding specific numeric constraints to the affected requirements. No requirement needs to be rewritten from scratch.

4.3 Sign-Off

Role	Name	Date
Prepared By	Systems Engineering Team	May 2026
Reviewed By	QA Lead	May 2026
Approved By	Project Sponsor	May 2026